Exercise-01: Process ID

File name: **"pid.c"**

Source code:

```
#include <stdio.h>
#include <unistd.h>

int main()
{
   int pid=getpid();
   printf("My Process ID (PID): %d\n", pid);

   int ppid=getppid();
   printf("Initial Parent PID (PPID): %d\n", ppid);

   return 0;
}
```

Output:

```
adil@ASUS-Vivobook-Intel-corei7:~/Lab_Report$ gcc -o pid pid.c
adil@ASUS-Vivobook-Intel-corei7:~/Lab_Report$ ./pid
My Process ID (PID): 3452
Initial Parent PID (PPID): 632
```

→If parent process terminated:

- When I terminate its parent, the child process becomes an Orphan Process.
- If I kill the parent process while the child is still running, the program will **not** stop running.
- Instead, the output will suddenly change. There can be seen the "Current Parent PID" switch to a new number, typically **1** or the PID of the system manager (like systemd).

Exercise-02: Variable's Memory Allocation Check

File name: **"memory.c"**

Source Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <math.h>

int global;
```

```
void check_memory_segment(int *ptr)
{
   uintptr_t target=(uintptr_t)ptr;

   int stack=10;
   uintptr_t stack_addr=(uintptr_t)&stack;

   int *heap=malloc(sizeof(int));
   uintptr_t heap_addr=(uintptr_t)heap;

   uintptr_t global_addr=(uintptr_t)&global;

   double dist_stack=fabs((double)target-(double)stack_addr);
   double dist_heap=fabs((double)target-(double)heap_addr);
   double dist_global=fabs((double)target-(double)global_addr);

   if (dist_stack<dist_heap && dist_stack<dist_global)
   {
      printf("STACK Segment\n");
   }
   else if(dist_heap<dist_stack && dist_heap<dist_global)
   {
      printf("HEAP Segment\n");
   }
   else
   {
      printf("GLOBAL Segment\n");
   }
}

int main()
{
   int var1=64;
   static int var2=20;
   int *var3=malloc(sizeof(int));

   printf("%p in ",&var1);
   check_memory_segment(&var1);

   printf("%p in ",&var2);
   check_memory_segment(&var2);

   printf("%p ",&var3);
   check_memory_segment(var3);
   return 0;
}
```

Output:

```
adil@ASUS-Vivobook-Intel-corei7:~/Lab_Report$ gcc -o mem memory.c
adil@ASUS-Vivobook-Intel-corei7:~/Lab_Report$ ./mem
0x7ffd1edb1d9c in STACK Segment
0x5dd962467010 in GLOBAL Segment
0x7ffd1edb1da0 HEAP Segment
adil@ASUS-Vivobook-Intel-corei7:~/Lab_Report$
```

Exercise-03: Command Line Arguments

File name: **"cla.c"**

Source Code:

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

char to_lower(char c)
{
   if(c>='A' && c<='Z')
   {
      return c+32;
   }
   return c;
}

int main(int argc, char *argv[])
{
   // String Reversing:
   int len=(int)strlen(argv[1]);
   char str[len+1];
   for(int i=0;i<len;i++)
   {
      str[i]=argv[1][i];
   }
   str[len]='\0';
   char str_rev[len+1];
   for(int i=0;i<len;i++)
   {
      str_rev[len-1-i]=str[i];
   }
   str_rev[len]='\0';
   printf("Reverse of %s is %s\n",str,str_rev);

   // Vowel-Consonant Check:
   len=(int)strlen(argv[2]);
```

```
    char str2[len+1];
    for(int i=0;i<len;i++)
    {
        str2[i]=argv[2][i];
    }
    str2[len]='\0';
    int vowel=0;
    for(int i=0;i<len;i++)
    {
        char c=to_lower(str2[i]);
        if(c=='a'||c=='e'||c=='i'||c=='o'||c=='u')
        {
            vowel++;
        }
    }
    printf("%s has %d vowels and %d consonants\n",str2,vowel,len-vowel);

    // addition and multiplication
    len=(int)strlen(argv[3]);
    char str3[len+1];
    for(int i=0;i<len;i++)
    {
        str3[i]=argv[3][i];
    }
    str3[len]='\0';
    len=(int)strlen(argv[4]);
    char str4[len+1];
    for(int i=0;i<len;i++)
    {
        str4[i]=argv[4][i];
    }
    str4[len]='\0';
    float a=atof(str3);
    float b=atof(str4);
    printf("the addition of %.2f and %.2f is %.2f\n",a,b,a+b);
    printf("the multiplication of %.2f and %.2f is %.2f\n",a,b,a*b);
}
```

Output:

```
adil@ASUS-Vivobook-Intel-corei7:~/Lab_Report$ gcc -o cla cla.c
adil@ASUS-Vivobook-Intel-corei7:~/Lab_Report$ ./cla hello Umbrella 15 9
Reverse of hello is olleh
Umbrella has 3 vowels and 5 consonants
the addition of 15.00 and 9.00 is 24.00
the multiplication of 15.00 and 9.00 is 135.00
adil@ASUS-Vivobook-Intel-corei7:~/Lab_Report$
```

Exercise-04: Working with Fork ()

File name: **“fork.c”**

Source Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
   char *filename="data.txt";
   int pid=fork();

   if(pid<0)
   {
      perror("Fork failed");
      return 1;
   }

   // --- Child Process ---
   if(pid==0)
   {
      printf("Child is Opening file to write...\n");

      FILE *fp=fopen(filename,"w");
      if (fp==NULL)
      {
         perror("Child failed to open file");
         exit(1);
      }

      fprintf(fp, "Hello World!! This message was written by the Child Process.");
      printf("Child has written the Data. Closing file.\n");

      fclose(fp);
      exit(0);// Terminate child
   }

   // --- Parent Process ---
   else
   {
      printf("Parent is Waiting...\n");
      // SYNCHRONIZATION: The parent halts here until the child calls exit()
      wait(NULL);
      printf("Child finished. Parent is Opening file to read...\n");
```

```
        // Open file in Read mode ("r")
        FILE *fp = fopen(filename, "r");
        if(fp==NULL)
        {
            perror("Parent failed to open file");
            return 1;
        }

        char text[100];
        // Read the content
        if(fgets(text, sizeof(text),fp)!=NULL)
        {
            printf("Parent is Read from file: \"%s\"\n", text);
        }

        fclose(fp);
    }
    return 0;
}
```

Output:

```
adil@ASUS-Vivobook-Intel-corei7:~/Lab_Report$ gcc -o fork fork.c
adil@ASUS-Vivobook-Intel-corei7:~/Lab_Report$ ./fork
Parent is Waiting...
Child is Opening file to write...
Child has written the Data. Closing file.
Child finished. Parent is Opening file to read...
Parent is Read from file: "Hello World!! This message was written by the Child Process."
adil@ASUS-Vivobook-Intel-corei7:~/Lab_Report$
```

Exercise-05: Semaphore and Race Condition

File name: **"sem.c"**

Source Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/mman.h>
#include <semaphore.h>

#define WITH_LOCK 0
#define PROCESS_COUNT 4
```

```
#define LOOPS_PER_CHILD 500

// Define a structure for variables we want to share across processes
typedef struct {
   int counter;
   sem_t mutex; // Semaphore to act as a lock
} SharedData;

int main()
{
   // 1. Create Shared Memory using mmap
   // We map a memory block that is readable, writable, and shared between processes
   // take different kinds of permission from os
   SharedData *shared=mmap(NULL, sizeof(SharedData),
                           PROT_READ | PROT_WRITE,
                           MAP_SHARED | MAP_ANONYMOUS, -1, 0);

   if(shared==MAP_FAILED)
   {
      perror("map failed");
      exit(1);
   }

   shared->counter=0;

   // Initialize Semaphore:
   // 2nd arg (1) = Shared between processes
   // 3rd arg (1) = Initial value (unlocked)
   if(sem_init(&shared->mutex,1,1)<0)
   {
      perror("Semaphore init failed");
      exit(1);
   }
   printf("--- Starting Simulation ---\n");
   printf("Mode: %s\n", WITH_LOCK ? "SAFE (Locked)" : "UNSAFE (Race Condition)");
   printf("Expected Final Count: %d\n", PROCESS_COUNT * LOOPS_PER_CHILD);

   // 3. Fork Processes
   for(int i=0;i<PROCESS_COUNT;i++)
   {
      int pid=fork();
      if(pid==0)
      {
         // --- CHILD PROCESS CODE ---
         for(int j=0;j<LOOPS_PER_CHILD;j++)
         {
```

```
            // [A] ENTRY SECTION (Lock)
            if(WITH_LOCK)
            {
               sem_wait(&shared->mutex);
            }

            // [B] CRITICAL SECTION (The Race happens here)
            int temp=shared->counter;

            // Artificial delay to force context switching
            // This makes the race condition happen much more often!
            usleep(1);
            shared->counter=temp+1;

            // [C] EXIT SECTION (Unlock)
            if(WITH_LOCK)
            {
               sem_post(&shared->mutex);
            }
         }
         exit(0); // Child finishes
      }
   }
   // 4. Parent waits for all children
   for(int i=0;i<PROCESS_COUNT;i++)
   {
      wait(NULL);
   }
   // 5. Results
   printf("--- Simulation Finished ---\n");
   printf("Final Counter Value: %d\n", shared->counter);

   if(shared->counter==PROCESS_COUNT*LOOPS_PER_CHILD)
   {
      printf("Result: SUCCESS (Data Consistent)\n");
   }
   else
   {
      printf("Result: FAILED (Race Condition Detected! Lost %d updates)\n",
            (PROCESS_COUNT * LOOPS_PER_CHILD) - shared->counter);
   }
   // Cleanup
   sem_destroy(&shared->mutex);
   munmap(shared, sizeof(SharedData));
   return 0;
}
```

Output:

```
adil@ASUS-Vivobook-Intel-corei7:~/Lab_Report$ gcc -o sem sem.c
adil@ASUS-Vivobook-Intel-corei7:~/Lab_Report$ ./sem
--- Starting Simulation ---
Mode: UNSAFE (Race Condition)
Expected Final Count: 2000
--- Simulation Finished ---
Final Counter Value: 503
Result: FAILED (Race Condition Detected! Lost 1497 updates)
```

If WITH_LOCK=1:

```
adil@ASUS-Vivobook-Intel-corei7:~/Lab_Report$ gcc -o sem sem.c
adil@ASUS-Vivobook-Intel-corei7:~/Lab_Report$ ./sem
--- Starting Simulation ---
Mode: SAFE (Locked)
Expected Final Count: 2000
--- Simulation Finished ---
Final Counter Value: 2000
Result: SUCCESS (Data Consistent)
adil@ASUS-Vivobook-Intel-corei7:~/Lab_Report$
```

Exercise-06: Child For Each Operation

File name: **"opr.c"**

Source Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
   float a,b;
   int fd[2];
   pipe(fd);
   printf("Enter two numbers: ");
   scanf("%f%f",&a,&b);
   printf("%.2f %.2f\n",a,b);

   for(int i=1;i<=2;i++)
   {
      int pid=fork();
      if(pid==0)
      {
         // child process
         // close(fd[0]);
         float result;
         if(i==1) //addition
         {
```

```
            result=a+b;
        }
        if(i==2) //subtraction
        {
            result=a-b;
        }
        write(fd[1],&result,sizeof(float));
        exit(0);
    }
    else
    {
        wait(NULL);
        float result;
        read(fd[0],&result,sizeof(float));
        if(i==1)
        {
            printf("Addition Result: %.2f\n", result);
        }
        if(i==2)
        {
            printf("Subtraction Result: %.2f\n", result);
        }
    }
  }
  close(fd[0]);
  close(fd[1]);
  return 0;
}
```

Output:

```
adil@ASUS-Vivobook-Intel-corei7:~/Lab_Report$ gcc -o opr opr.c
adil@ASUS-Vivobook-Intel-corei7:~/Lab_Report$ ./opr
Enter two numbers: 50.26 38.45
50.26 38.45
Addition Result: 88.71
Subtraction Result: 11.81
adil@ASUS-Vivobook-Intel-corei7:~/Lab_Report$
```

Conclusion:

In this series of experiments, I explored fundamental Operating System concepts and low-level C programming techniques. The tasks progressed from analyzing the memory layout of a single process to managing communication and synchronization between multiple processes. I have learned Memory Layout and Management Through the memory segment identification problem, Command Line Interfaces, Process Creation and Management using the fork () system call, Concurrency and Race Conditions--one of the most critical lessons came from the shared memory counter experiment, Inter-Process Communication (IPC).